



BSOS

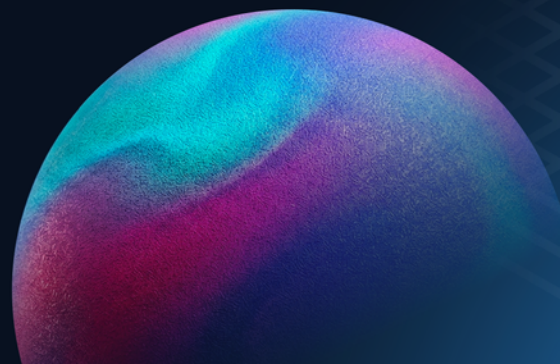
智能合約安全檢測報告

檢測工具

Slither

檢測日期

2026-08-13 ~ 2026-08-19



內部工作版本 — 不可作為交付文件

本報告尚有未完成處理或待確認之項目，僅供工程團隊追蹤使用；請於完成處理後重新產出報告，方可交付。

目錄

- 摘要
- 掃描範圍
- 協定理解摘要
- 協定概要
- 資產托管地圖
- 特權角色權限
- 信用風險的本質
- 檢測環境的一項偏離（必須揭露）
- 檢測方法
- 掃描環境資訊
- 情境庫覆蓋
- 待處理項目
- 發現明細
- 已評估項目摘要
- 附錄：發現處置分類

摘要

受檢對象：Isle Labs
檢測期間：2026-08-13 ~ 2026-08-19
受檢版本：3e438e336fe53050d0921a221dc43519cb13c46c

本次共提出 115 項發現（掃描工具產出 104 項、人工複核產出 11 項），依嚴重程度分布如下：

嚴重程度	筆數
Critical	0
High	5
Medium	26
Low	54
Informational	30
總計	115

揭露：上述筆數中，14 筆由風格預分類自動判定（naming-convention 11 筆、unindexed-event-address 3 筆），非人工逐筆判讀。

需要決策或行動的項目彙整於「待處理項目」，逐筆說明見「發現明細」。

掃描範圍

本次檢測涵蓋以下 33 個 Solidity 原始檔（合計 4542 行）。此清單即本報告的效力邊界——未列於此的檔案不在本次檢測範圍內。

檔案	行數
contracts/IsleGlobals.sol	125
contracts/LoanManager.sol	970
contracts/LoanManagerStorage.sol	27
contracts/Pool.sol	372
contracts/PoolAddressesProvider.sol	204
contracts/PoolConfigurator.sol	415
contracts/PoolConfiguratorStorage.sol	21
contracts/Receivable.sol	119
contracts/ReceivableStorage.sol	20
contracts/WithdrawalManager.sol	483
contracts/WithdrawalManagerStorage.sol	15
contracts/abstracts/Governable.sol	65
contracts/interfaces/IGovernable.sol	55
contracts/interfaces/IIsleGlobals.sol	118
contracts/interfaces/IIsleGlobalsEvents.sol	52
contracts/interfaces/ILoanManager.sol	115
contracts/interfaces/ILoanManagerEvents.sol	87
contracts/interfaces/ILoanManagerStorage.sol	49
contracts/interfaces/IPool.sol	92
contracts/interfaces/IPoolAddressesProvider.sol	121
contracts/interfaces/IPoolConfigurator.sol	172
contracts/interfaces/IPoolConfiguratorEvents.sol	77
contracts/interfaces/IPoolConfiguratorStorage.sol	34
contracts/interfaces/IReceivable.sol	37
contracts/interfaces/IReceivableEvent.sol	27
contracts/interfaces/IWithdrawalManager.sol	165
contracts/interfaces/IWithdrawalManagerStorage.sol	23
contracts/libraries/Errors.sol	198
contracts/libraries/PoolDeployer.sol	21
contracts/libraries/ReentrancyGuard.sol	40
contracts/libraries/types/DataTypes.sol	104
contracts/libraries/upgradability/UUPSProxy.sol	8
contracts/libraries/upgradability/VersionedInitializable.sol	111

檔案內容雜湊（SHA-256），供比對交付程式碼是否與受檢版本一致：

contracts/IsleGlobals.sol — 3987b303f8f62cd4848bb4a6bf2bad8a090fd51cbb0e12b0370e157d087da756
contracts/LoanManager.sol — 3b5220d68d3590065fd092f055a48092a4bd5d3096a32e695667e17c25637694
contracts/LoanManagerStorage.sol — 03f8c6932a04722f73a440df1fc7f8229053f3ed6f65fd5bfd8f290e2ad6e20
contracts/Pool.sol — 3ebb28118848e0b390ebe88568795a9fc4e0cb88c0c958695a692680664a5d97
contracts/PoolAddressesProvider.sol — fcb12e5f417360315c0efde4768b00a8a7c0e4052fb78e0ad6030b477e7fc055

contracts/PoolConfigurator.sol — 72c5c0db996978e18d8cb4e0ea477bbdc288930beac5a6211a602240709bc9b1

contracts/PoolConfiguratorStorage.sol — a06ef12092ffde999b82bd7063be4a4292eb4b55f0607aa767ca9840cc1b6a55

contracts/Receivable.sol — 3326392b70ffe4b8916b94d538a26c2be3b8fdf954e0468e04abbb127010c914

contracts/ReceivableStorage.sol — 80005aceeaae4aa704ff712cfc7ad92c1fdc02ca47716f25c8bf46831a1c127a

contracts/WithdrawalManager.sol — 6e549041783fbb4adc01e47987e634ad4d19e25211298683f6a508d1d6dcd48a

contracts/WithdrawalManagerStorage.sol — 5dd442b2a98144446e91cdcc04951c32bd56792c3e5cd91c0b7d09df16038bd3

contracts/abstracts/Governable.sol — c23132fae93d3293dc45cfef0bc5c7b214d130a7457c33ad628208fb47ef7f3f

contracts/interfaces/IGovernable.sol — 15e42654c06eba75b5fd08c150e91fbfc93e3141da950cf8b827acaaca60a6bc

contracts/interfaces/IIsleGlobals.sol — 3c18df7a4840be63b81d4ba30f50be6a3e6b3b65e6b943bc723dc43309ea7080

contracts/interfaces/IIsleGlobalsEvents.sol — b839a4dd2428e859daadf3189d61bbb7824ed3587de3930b9b6fd15d32dcf3e4

contracts/interfaces/ILoanManager.sol — 1b77aa8718d9a7f29bc75daa23519898df2cfe42ccf0369aef713be816ed95fe

contracts/interfaces/ILoanManagerEvents.sol — fd61b350a2b3237d3b4c57c711cda24b2a12ca680bd4960e3e9ab367918814a0

contracts/interfaces/ILoanManagerStorage.sol — 62f152636716b1f29bf056d50e99fe4366d1100a65b2203be25947d0e734acc6

contracts/interfaces/IPool.sol — 80df7b35418623f35460a31d87ebc3c5555cafc0799a05be4fd23e821ebab5f1

contracts/interfaces/IPoolAddressesProvider.sol — c16f2a9a3fe1db98bca4801d0b89d8e38358c8e7c63d2e869453237b2d1cae04

contracts/interfaces/IPoolConfigurator.sol — 96da95b1e7aced3884198db2251b68e7b7cb77816bc2dc3e0e884ad2845ab9f7

contracts/interfaces/IPoolConfiguratorEvents.sol — 4174280c1d83cdacdf042889ffef2fd033edc611e8afae0249b6ba7999e6aaf

contracts/interfaces/IPoolConfiguratorStorage.sol — c1e231d1db62fb9e9a3e037a3abadde839e4bf09289c8161c1fb1bfff055a0558

contracts/interfaces/IReceivable.sol — 30b8d71441367a4d4ffe4bf72163832048148f94c58900f5870d1e753d96ca16

contracts/interfaces/IReceivableEvent.sol — 4c5af0d433d5182fb9e6022f3ae1cb36f5c6198cc9f004df1b421baa5abcd10c

contracts/interfaces/IWithdrawalManager.sol — f02ba4ea6e70accb978e52e7dfb517677f8ae61a3056a09db046a1ff56fcd3d2

contracts/interfaces/IWithdrawalManagerStorage.sol — 4feb91d731a3524d37715be4db469a45895a7d8919aaaeb5ad3f80cd2da091c4

contracts/libraries/Errors.sol — fdc920a2a5ea9dbc0f7320e33c8cdec7c786803028ed76b83ef6ee1eb2f2151

contracts/libraries/PoolDeployer.sol — ecff7ef85239a5ee36c8da5d3e8be4d8f572d1f2ef7ba8a0ae8218b36e742588

contracts/libraries/ReentrancyGuard.sol — c2fc2d7b7d40a86c119a054c373f3ce1d97d2c2ccffc34a9e495e1219cb5ed99

contracts/libraries/types/DataTypes.sol — 03219ee7db03442b5935445e3fb166ab67f792d1fa9914cc33532900e1fef67b

contracts/libraries/upgradability/UUPSProxy.sol — 080abfe3e2d23e9c5447d5dea5dc0f7be5775603482a15526138571630af6679

contracts/libraries/upgradability/VersionedInitializable.sol — 5c4636c308adb1c0b0f4abcc0888471976a53f8fdacf294a8640c945a805631

協定理解摘要

協定概要

Isle v1 是應收帳款融資協定：買方（borrower）以短天期應收帳款憑證向資金池借款，存款人（lender）提供資金賺取利息。程式碼衍生自 Maple v2，為逐檔手工改寫而非 fork。

本次檢測範圍為 contracts/ 目錄下 33 個 Solidity 檔案、共 4,542 行，不含 modules/ 底下的相依套件（OpenZeppelin v4.9.2、forge-std、prb-math）。

核心合約分工：

合約	職責
Pool	ERC-4626 份額金庫，持有閒置流動性，負責存款與贖回的份額換算
PoolConfigurator	市場層設定與資金調度樞紐，持有第一損失準備（pool cover）
LoanManager	貸款生命週期與利息帳務，撥款後短暫持有資金
WithdrawalManager	以「週期 + 視窗」排隊處理贖回，持有排隊中的份額憑證
Receivable	應收帳款憑證（ERC-721）
IsleGlobals	全域白名單、協定費率、暫停開關
PoolAddressesProvider	各核心合約的位址註冊與代理升級入口

資產托管地圖

系統流通兩種資產：**pool asset**（ERC-20，實務上為 USDC，由 governor 白名單控管）與 **receivable**（ERC-721 應收帳款憑證）。

pool asset 在生命週期中會停留在四個位置：

停放位置	何時有餘額	誰能移動
Pool	存款人存入後、尚未放款前的閒置流動性	PoolConfigurator（撥款）與 Pool 自身（贖回）
LoanManager	撥款後、借款人提領前；以及還款進出的同一筆交易內	持有對應 receivable 憑證的人；合約自身的分配邏輯
PoolConfigurator	第一損失準備（pool cover）	pool admin
借款人指定地址	提領之後	已離開協定

兩點值得存款人注意：

1. Pool 在部署時即對 PoolConfigurator 授出**無上限且不可撤銷**的 ERC-20 授權。因此「誰能動資金池裡的錢」實際上等於「誰被註冊為 loan manager」，而該註冊由 governor 控制。
2. depositCover 開放任何人存入第一損失準備，但只有 pool admin 能提領。這是單向的資金投入。

receivable 憑證的流向為：建立後鑄造給賣方 → 賣方以憑證換取資金（憑證轉入 LoanManager）→ 買方清償後銷毀。憑證的內容（買方、賣方、面額、到期日）在建立時由呼叫者自填，合約端不做真實性驗證。

特權角色權限

系統有兩個具名特權角色。

Governor

單一位址，透過提名／接受兩段式移轉。合約層不強制多簽或時間鎖。權限涵蓋：

- 替換任一核心合約的實作（升級），或直接改寫核心合約的註冊位址
- 增刪全部白名單（可用資產、可用憑證、pool admin 名單）
- 設定協定費率、全域與單一函式的暫停開關
- 指派與更換 pool admin

Governor 同時具備 pool admin 的全部權限。**這個角色的權限邊界涵蓋資金池的全部資產**，其私鑰管理方式（多簽門檻、時間鎖、金鑰保管）屬於部署與營運層的控制，不在程式碼檢測範圍內，建議另行向存款人揭露。

Pool Admin

由 governor 指派，且必須在白名單內。權限涵蓋：

- **決定是否撥款** —— 這是唯一的信用審核關卡；合約層不對額度、利率、天期做任何合理性檢查
- 提領第一損失準備（受最低準備門檻限制）
- 設定管理費率、贖回週期參數、交易對手白名單、是否開放公眾存款
- 認列與撤銷貸款減值、觸發違約

無權限函式

以下函式任何位址皆可呼叫：建立應收帳款憑證、存入第一損失準備、推進利息帳務、代為還款。存款與贖回受白名單或公開開關限制。

信用風險的本質

本協定承作的是**無實體擔保**的短天期放款，借款人的償付能力由 pool admin 於鏈下審核。合約層不存在擔保率、清算價格或自動平倉機制；貸款逾期後的處理路徑為「認列減值 → 觸發違約 → 動用第一損失準備」，不足部分由存款人承擔。這是本類協定的共同設計取舍，非程式缺陷，但它決定了本報告中與「特權角色參數無上下限」相關的發現為何重要——在沒有自動化風控的系統裡，人工設定的參數就是最後一道防線。

檢測環境的一項偏離（必須揭露）

靜態分析工具 Slither 0.11.4 **無法直接解析本專案原始碼**，錯誤為

Type not found struct WithdrawalManager.CycleConfig。根因是 contracts/libraries/types/DataTypes.sol 內的四個 library（PoolConfigurator、WithdrawalManager、Receivable、Loan）與同名 contract 衝突，工具的名稱解析會解到 contract 上。

為完成本次掃描，檢測方在本機**暫時**將這四個 library 更名為 *_Types 並以 import alias 接回原名，掃描結束後立即還原；本次交付不含任何合約改動。此改名為純識別字重新命名，不改變任何邏輯、控制流或儲存佈局，也不改變行數，因此本報告引用的所有「檔案:行號」對原始碼仍然成立。副作用有兩處：一是「掃描範圍」章節記錄的檔案雜湊值中，有 7 個檔案是改名後的值，與原始碼不符；二是工具輸出的函式簽章中，型別名以改名後的形式出現（Loan_Types.PaymentInfo、Receivable_Types.Create 等），原始碼中的對應名稱為 Loan.PaymentInfo、Receivable.Create。兩者皆不影響發現本身的內容與行號。

建議儘速修正此命名衝突。在修正之前，本專案無法被 Slither 掃描——若 CI 中已配置靜態分析，它會回報「零發現」而非失敗，等同於靜態分析形同虛設。本次首輪掃描即出現此現象：工具回傳 success: false 但流程仍以「無發現」結束。

檢測方法

本次檢測依以下步驟執行：

1. **建置與環境確認**：確認專案可完整編譯，記錄工具鏈與相依套件版本（見「掃描環境資訊」）。

2. **靜態分析掃描**：以 Slither 對「掃描範圍」所列全部原始檔執行完整 detector 掃描。

3.

逐筆人工分類複核：對掃描產出的每一筆發現判定其處置分類（A／B／C／D）並記錄判斷依據；工具回報的嚴重度與本報告呈現的嚴重度若有落差，逐筆附上調整理由。

4.

情境式邏輯漏洞比對：對範圍內每一份合約，逐條比對內部維護的邏輯漏洞情境庫（權限檢查實作、未保護的狀態變更、旗標未落實、價格源可操縱、記帳與實際結果脫鉤、簽章雜湊綁定範圍、可組合模組的交互失效等），補靜態分析無法涵蓋的業務邏輯層級問題。逐合約的比對結果見「情境庫覆蓋」。

5.

領域事故模式比對：依受檢系統所屬業務領域，比對該領域公開已知的事故模式，檢查應具備而未實作的機制。

6. **產出與覆核**：彙整為本報告，並對共用同一判斷理由的發現群組進行抽查。

範圍限制

- 靜態分析擅長偵測「程式寫法特徵層級」的問題（重入模式、tx.origin 授權、弱亂數來源、未檢查的低階呼叫回傳值等）；業務邏輯層級的問題不在其可偵測範圍內，由上述第 4、5 步以人工方式補足，但人工複核的覆蓋程度不等同於系統性審計。
- 本報告為交付前之**自我檢查證明**，證明工程團隊已執行掃描並對每一筆發現完成逐筆判讀；其不構成、亦不取代由獨立第三方執行之完整安全審計。

掃描環境資訊

項目	內容
掃描時間	2026-08-19T01:10:01.970858
專案路徑	/Users/kai/BSOS/isle-enterprise-contract
Git commit	3e438e336fe53050d0921a221dc43519cb13c46c
Solidity / solc 版本	0.8.24
Slither 版本	0.11.4
Foundry (forge) 版本	forge Version: 1.7.1

情境庫覆蓋

下表為情境庫逐合約的比對結果。「已查證」為前置條件成立、實際讀碼確認過的情境數；「不適用」為合約不具備該情境前置條件而跳過的情境數；「命中」列出對應的發現編號。

合約	已查證	不適用	命中
contracts/LoanManager.sol	10	9	ISL-105、ISL-106、ISL-107、ISL-110、ISL-111、ISL-02
contracts/PoolConfigurator.sol	10	9	ISL-108、ISL-05
contracts/Pool.sol	10	9	—
contracts/WithdrawalManager.sol	8	11	ISL-112
contracts/Receivable.sol	7	12	ISL-109、ISL-115
contracts/IsleGlobals.sol	7	12	ISL-107
contracts/PoolAddressesProvider.sol	6	13	ISL-113、ISL-114
contracts/abstracts/Governable.sol	5	14	ISL-113
contracts/libraries/upgradability/VersionedInitializable.sol	4	15	ISL-114
contracts/libraries/ReentrancyGuard.sol	5	14	—
contracts/libraries/PoolDeployer.sol	2	17	—
contracts/libraries/upgradability/UUPSProxy.sol	2	17	—

待處理項目

狀態欄位反映本次產出報告當下的處置進度，尚未標註狀態的項目預設顯示「待處理」。

編號	標題	嚴重度	工具 impact	處置	狀態
ISL-105	利息帳務在「部分逾期」時重複入帳，永久虛增資產淨值	High	—	已確認需修復 (A)	待處理
ISL-02	reentrancy-eth	Medium	High	已確認需修復 (A)	待處理
ISL-03	reentrancy-eth	Medium	High	已確認需修復 (A)	待處理
ISL-05	unchecked-transfer	Medium	High	已確認需修復 (A)	待處理
ISL-106	accruedInterest() 缺少到期日上限，無交易期間份額價格無界虛增	Medium	—	已確認需修復 (A)	待處理
ISL-107	protocolFee + adminFee 合計無上限，可使fundLoan 永久revert	Medium	—	已確認需修復 (A)	待處理
ISL-108	maxCoverLiquidation 無上限，可使triggerDefault 永久revert	Medium	—	已確認需修復 (A)	待處理
ISL-109	Receivable.createReceivable 完全無存取控制	Medium	—	已確認需修復 (A)	待處理
ISL-110	借款人自訂的gracePeriod 無上限，可使triggerDefault 永久無法觸發	Medium	—	已確認需修復 (A)	待處理
ISL-112	setExitConfig 的cycleDuration 無上限，pool admin可實質凍結全部贖回	Medium	—	已確認需修復 (A)	待處理
ISL-114	升級的原子性完全依賴呼叫者傳入正確的params，否則initialize 對任何人開放	Medium	—	已確認需修復 (A)	待處理
ISL-111	同一張應收帳款可支撐多筆貸款，合約無「已使用」標記	Low	—	已確認需修復 (A)	待處理

發現明細

以下逐筆列出 33

項發現。低嚴重度且已判定為可接受風險或誤報的項目不在此節，彙整於「已評估項目摘要」。

ISL-01 | arbitrary-send-erc20

嚴重度	High
處置	誤報 (C)
位置	contracts/PoolConfigurator.sol:169-192

說明：PoolConfigurator.requestFunds(uint256) (contracts/PoolConfigurator.sol#169-192) uses arbitrary from in transferFrom: IERC20(asset_).safeTransferFrom(pool_,msg.sender,principal_) (contracts/PoolConfigurator.sol#185)

判斷依據：誤報。from 不是任意地址而是 pool 這個 storage 變數 (PoolConfigurator.sol:171、185)，授權來自 Pool 建構子對 configurator 的 approve(configurator, type(uint256).max) (Pool.sol:36)，屬協定內部的既定信任關係。呼叫者另受 PoolConfigurator.sol:175-177 的 msg.sender != loanManager_ 檢查限制。真正的風險不在此處的 from，而在 loanManager_ 來源可被 governor 改寫，已另立 ISL-113 追蹤。

ISL-04 | unchecked-transfer

嚴重度	High
處置	誤報 (C)
位置	contracts/WithdrawalManager.sol:235-285

說明：WithdrawalManager.processExit(uint256,address) (contracts/WithdrawalManager.sol#235-285) ignores return value by IERC20(_pool()).transfer(owner_,redeemableShares_) (contracts/WithdrawalManager.sol#264)

判斷依據：誤報。_pool() 回傳的是本協定自己由 PoolDeployer 部署的 Pool 合約 (PoolConfigurator.sol:104-105、109)，它繼承 OZ 的 ERC20，transfer 失敗時 revert、成功時固定回傳 true，不存在「回傳 false 而不 revert」的路徑。地址來源鏈為 _poolConfigurator().pool()，非外部可控。仍建議為一致性改用 safeTransfer。

ISL-06 | unprotected-upgrade

嚴重度	High
處置	誤報 (C)
位置	contracts/Receivable.sol:22-119

說明：Receivable (contracts/Receivable.sol#22-119) is an upgradeable contract that does not protect its initialize functions: Receivable.initialize(address) (contracts/Receivable.sol#49-56). Anyone can delete the contract with: UUPSUpgradeable.upgradeTo(address)

```
(modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol#68-71)UUPSUpgradeable.upgradeToAndCall(address,bytes)
```

```
(modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol#83-86)
```

判斷依據：誤報。Slither 的判定前提是「攻擊者可對 implementation 直接呼叫 initialize 取得控制權，再經 UUPS 升級路徑 selfdestruct 或改寫」。本專案的 OZ 版本為 v4.9.2，UUPSUpgradeable.upgradeTo 與 upgradeToAndCall 都帶 onlyProxy modifier (modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol:68、83)，直接對 implementation 呼叫時 address(this) == __self 會 revert —— 升級路徑不可達，攻擊鏈的第二步就斷了。已逐一確認兩份合約皆無其他 selfdestruct、delegatecall 或可提領資產的函式，implementation 本身也不持有任何資產。針對 Receivable 另外確認：implementation 上的 isleGlobal 為 0，_authorizeUpgrade 的 governor() (Receivable.sol:90-92) 會對 address(0) 外部呼叫而 revert，構成第二道阻擋。仍建議補上 constructor() { _disableInitializers(); } 作為縱深防禦。

ISL-07 | unprotected-upgrade

嚴重度	High
處置	誤報 (C)
位置	contracts/IsleGlobals.sol:12-125

說明：IsleGlobals (contracts/IsleGlobals.sol#12-125) is an upgradeable contract that does not protect its initialize functions: IsleGlobals.initialize(address) (contracts/IsleGlobals.sol#45-51). Anyone can delete the contract with: UUPSUpgradeable.upgradeTo(address)

```
(modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol#68-71)UUPSUpgradeable.upgradeToAndCall(address,bytes)
```

```
(modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol#83-86)
```

判斷依據：誤報。Slither 的判定前提是「攻擊者可對 implementation 直接呼叫 initialize 取得控制權，再經 UUPS 升級路徑 selfdestruct 或改寫」。本專案的 OZ 版本為 v4.9.2，UUPSUpgradeable.upgradeTo 與 upgradeToAndCall 都帶 onlyProxy modifier (modules/openzeppelin-contracts/contracts/proxy/utils/UUPSUpgradeable.sol:68、83)，直接對 implementation 呼叫時 address(this) == __self 會 revert —— 升級路徑不可達，攻擊鏈的第二步就斷了。已逐一確認兩份合約皆無其他 selfdestruct、delegatecall 或可提領資產的函式，implementation 本身也不持有任何資產。針對 IsleGlobals 另外確認：initializer 來自自製的 VersionedInitializable，在 implementation 自身 storage 上 lastInitializedRevision 確實為 0、可被任意人呼叫一次，但取得的只是一份沒有資產、沒有代理指向、且升級入口被 onlyProxy 擋住的孤兒 storage。仍建議在建構子鎖死 initializer。

ISL-105 | 利息帳務在「部分逾期」時重複入帳，永久虛增資產淨值

嚴重度	High
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:588-594

說明：_advanceGlobalPaymentAccounting() 的迴圈結束時只寫回 domainEnd 與 issuanceRate，漏了 domainStart。迴圈已用全額 issuance rate 把「舊 domainStart → 最後處理的到期日」整段記進帳（其中含尚未逾期 payment 的份額），第 593 行的 accruedInterest() 又用縮減後的 rate 配上尚未更新的 domainStart 把「舊 domainStart → 現在」整段再記一次，使未逾期 payment 在重疊區間被入帳兩次。每次觸發多記未逾期 payment 的 issuanceRate 總和 × (最後處理的到期日 - 上次結算時點)。多出來的利息進入 accountedInterest → assetsUnderManagement() → Pool.totalAssets() → 份額價格，且無任何沖銷機制 (_compareAndSubtractAccountedInterest 只防下溢)。

判斷依據：以 PoC 實測確認：tests/integration/concrete/loan-manager/poc/AccountingBugs.t.sol 的 test_PoC_Issue1_MixedLate_DoubleCountsInterest 在兩筆不同到期日的貸款、其中一筆逾期的情境下，正確值 2,301,369,860 對實際值 3,287,671,230，多記 986,301,370 (USDC 6 位小數)，量級恰等於 $\text{rateB} \times (\text{dueA} - \text{start})$ 的公式預測（誤差 2 wei 內）。對照 test_PoC_Issue1_AllLate_IsUnaffected：全部逾期時迴圈把 issuanceRate 歸零、accruedInterest() 於 :111 短路回傳 0，帳是乾淨的——只有混合狀態出錯，這也是它長期未被發現的原因。上游 Maple fixed-term-loan-manager 在同一位置寫回三個變數，本專案手工移植時漏抄 domainStart 這一行。LoanManager.sol:612-617 的註解把這個系統性缺陷誤述為「捨入誤差」，修復時應一併改寫。

建議：在 LoanManager.sol:589 之後補上 `domainStart = SafeCast.toUint48(domainStart_);`，並保留第 594 行末尾的 `domainStart = block.timestamp`（負責迴圈未進入的情形）。合約改動之外另需：以 upgradeToAndCall 原子升級（見 ISL-114）、在 migration 內以「所有存續 payment 的應計總和」即時重算並沖銷存量差額（不可寫死金額）、發專用事件供索引器下修 _prevRevenue。詳見 tests/integration/concrete/loan-manager/poc/README.md 第 5 節。

ISL-02 | reentrancy-eth

嚴重度	Medium (工具 impact : High)
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:252-289

嚴重度調整理由：工具判 High 係假設 asset 為任意 ERC20。本專案的 asset 受 IsleGlobals.isPoolAsset 白名單控管 (PoolConfigurator.sol:99)，目前部署為無 transfer hook 的 USDC，另兩個取得控制權的地址 (poolAdmin、isleVault) 皆為協定自有。故實際可利用性需要額外的治理層失誤配合，降為 Medium；但因為修法成本極低且同檔其他函式已有防護，仍列為必須修復。

說明：Reentrancy in LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289): External calls: - IERC20(asset).safeTransferFrom(msg.sender,address(this),principalAndInterest_) (contracts/LoanManager.sol#264) - _distributeClaimedFunds(loanId_,principal_,interest_) (contracts/LoanManager.sol#269) - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (modules/openzeppelin-contracts/contracts/token/ERC20/utils/SafeERC20.sol#122) - (success,returndata) = target.call{value: value}(data)

(modules/openzeppelin-contracts/contracts/utils/Address.sol#135) -
IERC20(asset_).safeTransfer(_pool(),principal_ + netInterest_) (contracts/LoanManager.sol#875) -
IERC20(asset_).safeTransfer(_poolAdmin(),adminFee_) (contracts/LoanManager.sol#876) -
IERC20(asset_).safeTransfer(_vault(),protocolFee_) (contracts/LoanManager.sol#877) External calls
sending eth: - _distributeClaimedFunds(loanId_,principal_,interest_) (contracts/LoanManager.sol#269)
- (success,returndata) = target.call{value: value}(data)
(modules/openzeppelin-contracts/contracts/utils/Address.sol#135) State variables written after the
call(s): - paymentIssuanceRate_ = _handlePaymentAccounting(loanId_)
(contracts/LoanManager.sol#277) - accountedInterest -=
SafeCast.toUint112(_min(accountedInterest,amount_)) (contracts/LoanManager.sol#617)
LoanManagerStorage.accountedInterest (contracts/LoanManagerStorage.sol#13) can be used in cross
function reentrancies: - LoanManager._advanceGlobalPaymentAccounting()
(contracts/LoanManager.sol#553-595) -
LoanManager._compareAndSubtractAccountedInterest(uint256) (contracts/LoanManager.sol#612-618)
- LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) -
LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -
LoanManagerStorage.accountedInterest (contracts/LoanManagerStorage.sol#13) -
LoanManager.assetsUnderManagement() (contracts/LoanManager.sol#115-117) -
LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -
LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -
LoanManager.updateAccounting() (contracts/LoanManager.sol#167-170) - delete paymentIdOf[loanId_]
(contracts/LoanManager.sol#280) LoanManagerStorage.paymentIdOf
(contracts/LoanManagerStorage.sol#20) can be used in cross function reentrancies: -
LoanManager._deletePayment(uint16) (contracts/LoanManager.sol#719-722) -
LoanManager._distributeClaimedFunds(uint16,uint256,uint256) (contracts/LoanManager.sol#855-878) -
LoanManager._handlePaymentAccounting(uint16) (contracts/LoanManager.sol#724-765) -
LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -
LoanManagerStorage.paymentIdOf (contracts/LoanManagerStorage.sol#20) -
LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -
LoanManager.triggerDefault(uint16) (contracts/LoanManager.sol#419-455) -
LoanManager.withdrawFunds(uint16,address) (contracts/LoanManager.sol#292-313) -
paymentIssuanceRate_ = _handlePaymentAccounting(loanId_) (contracts/LoanManager.sol#277) -
paymentWithEarliestDueDate = next_ (contracts/LoanManager.sol#837)
LoanManagerStorage.paymentWithEarliestDueDate (contracts/LoanManagerStorage.sol#10) can be
used in cross function reentrancies: - LoanManager._advanceGlobalPaymentAccounting()
(contracts/LoanManager.sol#553-595) - LoanManager._removePaymentFromList(uint256)
(contracts/LoanManager.sol#830-849) - LoanManager._updateIssuanceParams(uint256,uint112)
(contracts/LoanManager.sol#597-606) - LoanManagerStorage.paymentWithEarliestDueDate
(contracts/LoanManagerStorage.sol#10) - paymentIssuanceRate_ =
handlePaymentAccounting(loanId) (contracts/LoanManager.sol#277) - delete payments[paymentId_]
(contracts/LoanManager.sol#735) LoanManagerStorage.payments

(contracts/LoanManagerStorage.sol#22) can be used in cross function reentrancies: -
LoanManager._accountToEndOfPayment(uint256,uint256,uint256,uint256)
(contracts/LoanManager.sol#699-717) - LoanManager._advanceGlobalPaymentAccounting()
(contracts/LoanManager.sol#553-595) - LoanManager._deletePayment(uint16)
(contracts/LoanManager.sol#719-722) - LoanManager._distributeClaimedFunds(uint16,uint256,uint256)
(contracts/LoanManager.sol#855-878) - LoanManager._handlePaymentAccounting(uint16)
(contracts/LoanManager.sol#724-765) - LoanManager._updateIssuanceParams(uint256,uint112)
(contracts/LoanManager.sol#597-606) - LoanManager.impairLoan(uint16)
(contracts/LoanManager.sol#316-365) - LoanManagerStorage.payments
(contracts/LoanManagerStorage.sol#22) - LoanManager.triggerDefault(uint16)
(contracts/LoanManager.sol#419-455) - paymentIssuanceRate_ = _handlePaymentAccounting(loanId_)
(contracts/LoanManager.sol#277) - sortedPayments[next_].previous = previous_
(contracts/LoanManager.sol#841) - sortedPayments[previous_].next = next_
(contracts/LoanManager.sol#845) - delete sortedPayments[paymentId_]
(contracts/LoanManager.sol#848) LoanManagerStorage.sortedPayments
(contracts/LoanManagerStorage.sol#23) can be used in cross function reentrancies: -
LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -
LoanManager._removePaymentFromList(uint256) (contracts/LoanManager.sol#830-849) -
LoanManagerStorage.sortedPayments (contracts/LoanManagerStorage.sol#23)

判斷依據：成立。repayLoan (LoanManager.sol:252) 缺少 nonReentrant，而同一份合約的 fundLoan (:231) 與 removeLoanImpairment (:368) 都有 —— 這是遺漏而非設計。不變量破口確實存在：步驟 4 _distributeClaimedFunds 已把 principal 轉進 pool (:875)，步驟 5 才把 principalOut 減掉 (:273)，兩者之間 _totalAssets() (PoolConfigurator.sol:344 = pool 餘額 + AUM) 會重複計入該筆本金，份額價格短暫虛高。逐行確認取得控制權的途徑有二：(a) asset 具備 transfer hook (ERC777/ERC1363)；(b) _poolAdmin() 或 _vault() 為可執程式碼的地址 (:876-877)。現行 asset 由 governor 白名單控管且為 USDC，故實際可利用性低，但修法成本只有一個 modifier。

建議：在 repayLoan 加上 nonReentrant modifier（與 fundLoan、removeLoanImpairment 一致）。另建議把步驟 5 的 principalOut 遞減移到步驟 4 的資金分配之前，讓 _totalAssets() 在整筆交易期間都不會重複計入本金。

ISL-03 | reentrancy-eth

嚴重度	Medium (工具 impact : High)
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:252-289

嚴重度調整理由：同 ISL-02：同一函式、同一修法，降級理由一致。

說明：Reentrancy in LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289): External calls: - IERC20(asset).safeTransferFrom(msg.sender,address(this),principalAndInterest_)

(contracts/LoanManager.sol#264) - _distributeClaimedFunds(loanId_,principal_,interest_)
(contracts/LoanManager.sol#269) - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (modules/openzeppelin-contracts/contracts/token/ERC20/utils/SafeERC20.sol#122) -
(success,returndata) = target.call{value: value}(data)
(modules/openzeppelin-contracts/contracts/contracts/utils/Address.sol#135) -
IERC20(asset_).safeTransfer(_pool(),principal_ + netInterest_) (contracts/LoanManager.sol#875) -
IERC20(asset_).safeTransfer(_poolAdmin(),adminFee_) (contracts/LoanManager.sol#876) -
IERC20(asset_).safeTransfer(_vault(),protocolFee_) (contracts/LoanManager.sol#877) -
IReceivable(loan_.receivableAsset).burnReceivable(loan_.receivableTokenId)
(contracts/LoanManager.sol#285) External calls sending eth: -
distributeClaimedFunds(loanId,principal_,interest_) (contracts/LoanManager.sol#269) -
(success,returndata) = target.call{value: value}(data)
(modules/openzeppelin-contracts/contracts/contracts/utils/Address.sol#135) State variables written after the call(s): - _updateIssuanceParams(issuanceRate - paymentIssuanceRate_,accountedInterest)
(contracts/LoanManager.sol#288) - IssuanceParamsUpdated(domainEnd =
block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_)
(contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd =
payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest =
accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.accountedInterest
(contracts/LoanManagerStorage.sol#13) can be used in cross function reentrancies: -
LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -
LoanManager._compareAndSubtractAccountedInterest(uint256) (contracts/LoanManager.sol#612-618)
- LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) -
LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -
LoanManagerStorage.accountedInterest (contracts/LoanManagerStorage.sol#13) -
LoanManager.assetsUnderManagement() (contracts/LoanManager.sol#115-117) -
LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -
LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -
LoanManager.updateAccounting() (contracts/LoanManager.sol#167-170) -
updateIssuanceParams(issuanceRate - paymentIssuanceRate,accountedInterest)
(contracts/LoanManager.sol#288) - IssuanceParamsUpdated(domainEnd =
block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_)
(contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd =
payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest =
accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.domainEnd
(contracts/LoanManagerStorage.sol#12) can be used in cross function reentrancies: -
LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -
LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -
LoanManagerStorage.domainEnd (contracts/LoanManagerStorage.sol#12) -
updateIssuanceParams(issuanceRate - paymentIssuanceRate,accountedInterest)
(contracts/LoanManager.sol#288) - IssuanceParamsUpdated(domainEnd =

block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_) (contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd = payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest = accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.issuanceRate (contracts/LoanManagerStorage.sol#16) can be used in cross function reentrancies: - LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) - LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) - LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) - LoanManager.accruedInterest() (contracts/LoanManager.sol#109-112) - LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) - LoanManagerStorage.issuanceRate (contracts/LoanManagerStorage.sol#16) - LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) - LoanManager.updateAccounting() (contracts/LoanManager.sol#167-170)

判斷依據：成立。repayLoan (LoanManager.sol:252) 缺少 nonReentrant，而同一份合約的 fundLoan (:231) 與 removeLoanImpairment (:368) 都有 —— 這是遺漏而非設計。不變量破口確實存在：步驟 4 _distributeClaimedFunds 已把 principal 轉進 pool (:875)，步驟 5 才把 principalOut 減掉 (:273)，兩者之間 _totalAssets() (PoolConfigurator.sol:344 = pool 餘額 + AUM) 會重複計入該筆本金，份額價格短暫虛高。逐行確認取得控制權的途徑有二：(a) asset 具備 transfer hook (ERC777/ERC1363)；(b) _poolAdmin() 或 _vault() 為可執行程式碼的地址 (:876-877)。現行 asset 由 governor 白名單控管且為 USDC，故實際可利用性低，但修法成本只有一個 modifier。本筆與 ISL-02 為 Slither 對同一函式不同狀態變數各報一次，同一個修法一併涵蓋。

建議：同 ISL-02，單一 nonReentrant modifier 即同時涵蓋兩筆。

ISL-05 | unchecked-transfer

嚴重度	Medium (工具 impact : High)
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/PoolConfigurator.sol:382-392

嚴重度調整理由：工具判 High 係假設代幣行為未知。目前部署資產為標準回傳 true 的 USDC，兩條失敗路徑都需要 governor 先把非標準代幣加進 isPoolAsset 白名單才會觸發，不是任何外部人可直接觸發的資金損失，故降為 Medium。但這是一行的修法，且缺陷會同時影響資金正確性與違約流程的可用性，仍列為必須修復。

說明：PoolConfigurator._handleCover(uint256) (contracts/PoolConfigurator.sol#382-392) ignores return value by IERC20(asset).transfer(pool,coverAmount_) (contracts/PoolConfigurator.sol#389)

判斷依據：成立。_handleCover (PoolConfigurator.sol:389) 對 asset 使用原生 transfer 並忽略回傳值，而同一份合約其餘所有 asset 操作都走 SafeERC20 (:185、:247、:256) —— 明顯的遺漏。asset 是 governor 白名單的任意 ERC20，兩種常見代幣都會出問題：(a) 回傳 false 而不 revert 的代幣 → poolCover -= coverAmount_ (:387) 已先扣減，cover 帳面減少但資金沒動，差額無法回復；(b)

完全不回傳資料的代幣（USDT 型）→ OZ IERC20.transfer 解碼空回傳值會 revert，使 triggerDefault 全面卡死、無法認列違約。

建議：改用 IERC20(asset).safeTransfer(pool, coverAmount_)（本檔已 using SafeERC20 for IERC20，無需新增 import）。並建議把 poolCover -= coverAmount_ 移到轉帳成功之後。

ISL-08 | divide-before-multiply

嚴重度	Medium
處置	誤報 (C)
位置	contracts/LoanManager.sol:493-511

說明：LoanManager._getLateInterest(uint256,uint256,uint256,uint256,uint256) (contracts/LoanManager.sol#493-511) performs a multiplication on the result of a division: - $\text{fullDaysLate_} = ((\text{currentTime_} - \text{dueDate_} + (86400 - 1)) / 86400) * 86400$ (contracts/LoanManager.sol#508)

判斷依據：誤報。 $\text{fullDaysLate_} = ((\text{currentTime_} - \text{dueDate_} + 86399) / 86400) * 86400$ (LoanManager.sol:508) 是刻意的「逾期天數無條件進位到整日」語意，先除後乘正是取整的手段，不是精度損失。此行為與鏈上實測一致：postmortem 以此公式重算 53 筆還款利息，與 LoanRepaid 事件分毫不差。

ISL-09 | divide-before-multiply

嚴重度	Medium
處置	已知風險但可接受 (B)
位置	contracts/LoanManager.sol:767-790

說明：LoanManager._queuePayment(uint16,uint256,uint256) (contracts/LoanManager.sol#767-790) performs a multiplication on the result of a division: - $\text{newRate_} = (_getNetInterest(\text{interest_}, \text{feeRate_}) * \text{PRECISION}) / (\text{dueDate_} - \text{startDate_})$ (contracts/LoanManager.sol#775) - $\text{payments}[\text{paymentId_}] = \text{Loan_Types.PaymentInfo}(\{\text{protocolFee:SafeCast.toUint24}(\text{protocolFee_}), \text{adminFee:SafeCast.toUint24}(\text{adminFee_}), \text{startDate:SafeCast.toUint48}(\text{startDate_}), \text{dueDate:SafeCast.toUint48}(\text{dueDate_}), \text{incomingNetInterest:SafeCast.toUint128}(\text{newRate_} * (\text{dueDate_} - \text{startDate_}) / \text{PRECISION}), \text{issuanceRate:newRate_}\})$ (contracts/LoanManager.sol#780-787)

判斷依據：屬實但可接受。 $\text{newRate_} = _getNetInterest(\text{interest_}, \text{feeRate_}) * \text{PRECISION} / (\text{dueDate_} - \text{startDate_})$ (LoanManager.sol:775) 之中，interest_ 本身已在 _getInterest 內做過一次除法 (:490)，確實有先除後乘。但 $\text{PRECISION} = 1e27$ 的放大倍率遠大於任何實際天期，捨入誤差在 wei 級；這也是上游 Maple v2 的原始寫法，經多輪外部審計。真正需要處理的不是這裡的捨入，而是 _advanceGlobalPaymentAccounting 把系統性重複入帳誤判為捨入誤差（見 ISL-105 與 LoanManager.sol:612-617 的註解）。

ISL-10 | divide-before-multiply

嚴重度	Medium
-----	--------

處置	誤報 (C)
位置	contracts/LoanManager.sol:513-522

說明：LoanManager._getPeriodicInterestRate(uint256,uint256) (contracts/LoanManager.sol#513-522) performs a multiplication on the result of a division: - periodicInterestRate_ = (interestRate_ * (SCALED_ONE / HUNDRED_PERCENT) * interval_) / uint256(31536000) (contracts/LoanManager.sol#521)

判斷依據：誤報。SCALED_ONE / HUNDRED_PERCENT = 1e18 / 1e6 = 1e12，兩者皆為編譯期常數且整除，不產生任何餘數（LoanManager.sol:521）。Slither 只做語法比對，看不出被除數是常數。

ISL-11 | incorrect-equality

嚴重度	Medium
處置	誤報 (C)
位置	contracts/LoanManager.sol:633-665

說明：LoanManager._getDefaultInterestAndFees(uint16,Loan_Types.PaymentInfo) (contracts/LoanManager.sol#633-665) uses a dangerous strict equality: - grossLateInterest_ == 0 (contracts/LoanManager.sol#661)

判斷依據：誤報。Slither 的 incorrect-equality 針對的是「以嚴格等值比較餘額或時間戳」，本筆比較的是 grossLateInterest_ == 0 (LoanManager.sol:661)，用途是判別「這筆還款是否為逾期還款」以決定要不要按比例縮放管理費，語意上就是二值判斷，取值來自 _getLateInterest 的 currentTime_ <= dueDate_ → return 0 (:504-506)，不是餘額。

ISL-12 | incorrect-equality

嚴重度	Medium
處置	誤報 (C)
位置	contracts/WithdrawalManager.sol:463-469

說明：WithdrawalManager._emitProcess(address,uint256,uint256) (contracts/WithdrawalManager.sol#463-469) uses a dangerous strict equality: - sharesToRedeem_ == 0 (contracts/WithdrawalManager.sol#464)

判斷依據：誤報。Slither 的 incorrect-equality 針對的是「以嚴格等值比較餘額或時間戳」，本筆比較的是 sharesToRedeem_ == 0 (WithdrawalManager.sol:464)，純粹決定要不要發事件，無任何資金或狀態後果。

ISL-13 | incorrect-equality

嚴重度	Medium
處置	誤報 (C)
位置	contracts/LoanManager.sol:109-112

說明：LoanManager.accruedInterest() (contracts/LoanManager.sol#109-112) uses a dangerous strict equality: - issuanceRate_ == 0 (contracts/LoanManager.sol#111)

判斷依據：誤報。Slither 的 incorrect-equality 針對的是「以嚴格等值比較餘額或時間戳」，本筆比較的是 issuanceRate_ == 0 (LoanManager.sol:111)，用途是短路避免無謂運算。附帶說明：這一行在 postmortem 中是問題二的所在地，但缺陷是缺少 _min(block.timestamp, domainEnd) 上限，與這個等值比較無關；該缺陷已另立 ISL-106。

ISL-14 | incorrect-equality

嚴重度	Medium
處置	誤報 (C)
位置	contracts/WithdrawalManager.sol:292-300

說明：WithdrawalManager.isInExitWindow(address) (contracts/WithdrawalManager.sol#292-300) uses a dangerous strict equality: - exitCycleId_ == 0 (contracts/WithdrawalManager.sol#295)

判斷依據：誤報。Slither 的 incorrect-equality 針對的是「以嚴格等值比較餘額或時間戳」，本筆比較的是 exitCycleId_ == 0 (WithdrawalManager.sol:295)，0 是「無提領請求」的哨兵值 (processExit 於 :276 明確寫入 0)，不是可被外部操縱的數量。

ISL-15 | incorrect-equality

嚴重度	Medium
處置	誤報 (C)
位置	contracts/WithdrawalManager.sol:452-461

說明：WithdrawalManager._emitUpdate(address,uint256,uint256) (contracts/WithdrawalManager.sol#452-461) uses a dangerous strict equality: - lockedShares_ == 0 (contracts/WithdrawalManager.sol#453)

判斷依據：誤報。Slither 的 incorrect-equality 針對的是「以嚴格等值比較餘額或時間戳」，本筆比較的是 lockedShares_ == 0 (WithdrawalManager.sol:453)，決定發 WithdrawalCancelled 還是 WithdrawalUpdated，無資金後果。另已確認此處的早退同時保護了 getWindowAtId(0) 不被呼叫（該路徑會在 getConfigAtId underflow）。

ISL-16 | reentrancy-no-eth

嚴重度	Medium
處置	誤報 (C)
位置	contracts/LoanManager.sol:231-249

說明：Reentrancy in LoanManager.fundLoan(uint16) (contracts/LoanManager.sol#231-249): External calls: - IPoolConfigurator(_poolConfigurator()).requestFunds(principal_) (contracts/LoanManager.sol#238) State variables written after the call(s): - loanStorage_.startDate = block.timestamp (contracts/LoanManager.sol#242) LoanManagerStorage._loans

(contracts/LoanManagerStorage.sol#26) can be used in cross function reentrancies: -
LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) -
LoanManager.getLoanInfo(uint16) (contracts/LoanManager.sol#104-106) -
LoanManager.getLoanPaymentBreakdown(uint16) (contracts/LoanManager.sol#140-160) -
LoanManager.getLoanPaymentDetailedBreakdown(uint16) (contracts/LoanManager.sol#120-137) -
LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -
LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -
LoanManager.requestLoan(address,uint256,uint256,uint256,uint256[2])
(contracts/LoanManager.sol#173-228) - LoanManager.triggerDefault(uint16)
(contracts/LoanManager.sol#419-455) - LoanManager.withdrawFunds(uint16,address)
(contracts/LoanManager.sol#292-313) - loanStorage_.drawableFunds = principal_
(contracts/LoanManager.sol#243) LoanManagerStorage._loans (contracts/LoanManagerStorage.sol#26)
can be used in cross function reentrancies: - LoanManager._handleDefault(uint16,uint256,uint256)
(contracts/LoanManager.sol#884-913) - LoanManager.getLoanInfo(uint16)
(contracts/LoanManager.sol#104-106) - LoanManager.getLoanPaymentBreakdown(uint16)
(contracts/LoanManager.sol#140-160) - LoanManager.getLoanPaymentDetailedBreakdown(uint16)
(contracts/LoanManager.sol#120-137) - LoanManager.impairLoan(uint16)
(contracts/LoanManager.sol#316-365) - LoanManager.repayLoan(uint16)
(contracts/LoanManager.sol#252-289) -
LoanManager.requestLoan(address,uint256,uint256,uint256,uint256[2])
(contracts/LoanManager.sol#173-228) - LoanManager.triggerDefault(uint16)
(contracts/LoanManager.sol#419-455) - LoanManager.withdrawFunds(uint16,address)
(contracts/LoanManager.sol#292-313) - _updateIssuanceParams(issuanceRate +
queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)
(contracts/LoanManager.sol#248) - IssuanceParamsUpdated(domainEnd =
block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_)
(contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd =
payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest =
accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.accountedInterest
(contracts/LoanManagerStorage.sol#13) can be used in cross function reentrancies: -
LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -
LoanManager._compareAndSubtractAccountedInterest(uint256) (contracts/LoanManager.sol#612-618)
- LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) -
LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -
LoanManagerStorage.accountedInterest (contracts/LoanManagerStorage.sol#13) -
LoanManager.assetsUnderManagement() (contracts/LoanManager.sol#115-117) -
LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -
LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -
LoanManager.updateAccounting() (contracts/LoanManager.sol#167-170) -
_updateIssuanceParams(issuanceRate +
queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)

(contracts/LoanManager.sol#248) - IssuanceParamsUpdated(domainEnd = block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_)

(contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd = payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest = accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.domainEnd

(contracts/LoanManagerStorage.sol#12) can be used in cross function reentrancies: -

LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -

LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -

LoanManagerStorage.domainEnd (contracts/LoanManagerStorage.sol#12) -

_updateIssuanceParams(issuanceRate +

queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)

(contracts/LoanManager.sol#248) - IssuanceParamsUpdated(domainEnd = block.timestamp.toUint48(),issuanceRate = issuanceRate_,accountedInterest = accountedInterest_)

(contracts/LoanManager.sol#601-605) - IssuanceParamsUpdated(domainEnd = payments[earliestPayment_].dueDate,issuanceRate = issuanceRate_,accountedInterest = accountedInterest_) (contracts/LoanManager.sol#601-605) LoanManagerStorage.issuanceRate

(contracts/LoanManagerStorage.sol#16) can be used in cross function reentrancies: -

LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -

LoanManager._handleDefault(uint16,uint256,uint256) (contracts/LoanManager.sol#884-913) -

LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -

LoanManager.accruedInterest() (contracts/LoanManager.sol#109-112) -

LoanManager.impairLoan(uint16) (contracts/LoanManager.sol#316-365) -

LoanManagerStorage.issuanceRate (contracts/LoanManagerStorage.sol#16) -

LoanManager.repayLoan(uint16) (contracts/LoanManager.sol#252-289) -

LoanManager.updateAccounting() (contracts/LoanManager.sol#167-170) -

_updateIssuanceParams(issuanceRate +

queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)

(contracts/LoanManager.sol#248) - paymentWithEarliestDueDate = paymentId_

(contracts/LoanManager.sol#819) LoanManagerStorage.paymentWithEarliestDueDate

(contracts/LoanManagerStorage.sol#10) can be used in cross function reentrancies: -

LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -

LoanManager._removePaymentFromList(uint256) (contracts/LoanManager.sol#830-849) -

LoanManager._updateIssuanceParams(uint256,uint112) (contracts/LoanManager.sol#597-606) -

LoanManagerStorage.paymentWithEarliestDueDate (contracts/LoanManagerStorage.sol#10) -

_updateIssuanceParams(issuanceRate +

queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)

(contracts/LoanManager.sol#248) - payments[paymentId_] =

Loan_Types.PaymentInfo({protocolFee:SafeCast.toUint24(protocolFee_),adminFee:SafeCast.toUint24(adminFee_),startDate:SafeCast.toUint48(startDate_),dueDate:SafeCast.toUint48(dueDate_),incomingNetInterest:SafeCast.toUint128(newRate_ * (dueDate_ - startDate_) / PRECISION),issuanceRate:newRate_})

(contracts/LoanManager.sol#780-787) LoanManagerStorage.payments

(contracts/LoanManagerStorage.sol#22) can be used in cross function reentrancies: -
LoanManager._accountToEndOfPayment(uint256,uint256,uint256,uint256)
(contracts/LoanManager.sol#699-717) - LoanManager._advanceGlobalPaymentAccounting()
(contracts/LoanManager.sol#553-595) - LoanManager._deletePayment(uint16)
(contracts/LoanManager.sol#719-722) - LoanManager._distributeClaimedFunds(uint16,uint256,uint256)
(contracts/LoanManager.sol#855-878) - LoanManager._handlePaymentAccounting(uint16)
(contracts/LoanManager.sol#724-765) - LoanManager._updateIssuanceParams(uint256,uint112)
(contracts/LoanManager.sol#597-606) - LoanManager.impairLoan(uint16)
(contracts/LoanManager.sol#316-365) - LoanManagerStorage.payments
(contracts/LoanManagerStorage.sol#22) - LoanManager.triggerDefault(uint16)
(contracts/LoanManager.sol#419-455) - _updateIssuanceParams(issuanceRate +
queuePayment(loanId,block.timestamp,loan_.dueDate),accountedInterest)
(contracts/LoanManager.sol#248) - sortedPayments[current_].next = paymentId_
(contracts/LoanManager.sol#817) - sortedPayments[next_].previous = paymentId_
(contracts/LoanManager.sol#823) - sortedPayments[paymentId_] =
Loan_Types.SortedPayment({previous:current_,next:next_,paymentDueDate:paymentDueDate_})
(contracts/LoanManager.sol#826-827) LoanManagerStorage.sortedPayments
(contracts/LoanManagerStorage.sol#23) can be used in cross function reentrancies: -
LoanManager._advanceGlobalPaymentAccounting() (contracts/LoanManager.sol#553-595) -
LoanManager._removePaymentFromList(uint256) (contracts/LoanManager.sol#830-849) -
LoanManagerStorage.sortedPayments (contracts/LoanManagerStorage.sol#23)

判斷依據：誤報。fundLoan (LoanManager.sol:231) 本身帶 nonReentrant，Slither 不追蹤自製的 ReentrancyGuardUpgradeable (libraries/ReentrancyGuard.sol:30-39，ERC-7201 命名空間 storage) 因此無法辨識。已確認該 modifier 的 \$.status 讀寫邏輯正確。

ISL-17 | reentrancy-no-eth

嚴重度	Medium
處置	已知風險但可接受 (B)
位置	contracts/PoolAddressesProvider.sol:166-179

說明：Reentrancy in PoolAddressesProvider._updateImpl(bytes32,address,bytes)
(contracts/PoolAddressesProvider.sol#166-179): External calls: - proxy = new
TransparentUpgradeableProxy(newAddress,address(this),params)
(contracts/PoolAddressesProvider.sol#172) State variables written after the call(s): - _addresses[id] =
proxyAddress = address(proxy) (contracts/PoolAddressesProvider.sol#173)
PoolAddressesProvider._addresses (contracts/PoolAddressesProvider.sol#17) can be used in cross
function reentrancies: - PoolAddressesProvider._getProxyImplementation(bytes32)
(contracts/PoolAddressesProvider.sol#195-203) -
PoolAddressesProvider._updateImpl(bytes32,address,bytes)
(contracts/PoolAddressesProvider.sol#166-179) -
PoolAddressesProvider.constructor(string,IIslGlobals) (contracts/PoolAddressesProvider.sol#36-43) -

PoolAddressesProvider.getAddress(bytes32) (contracts/PoolAddressesProvider.sol#140-142) -
PoolAddressesProvider.setAddress(bytes32,address) (contracts/PoolAddressesProvider.sol#145-149) -
PoolAddressesProvider.setAddressAsProxy(bytes32,address,bytes)
(contracts/PoolAddressesProvider.sol#108-121) - PoolAddressesProvider.setIsleGlobals(address)
(contracts/PoolAddressesProvider.sol#133-137)

判斷依據：屬實但可接受。_updateImpl (PoolAddressesProvider.sol:166-179) 先 new TransparentUpgradeableProxy (:172, 建構子會 delegatecall 到 implementation 的 initialize) 才寫 _addresses[id] (:173)。重入需要 implementation 在 initialize 期間回呼 provider，而 implementation 是 governor 自己指定的合約，_updateImpl 全部入口皆 onlyGovernor。這屬於「governor 部署惡意 implementation」的既有信任範圍（見 ISL-113），不是額外的攻擊面。建議仍調整為先寫入再部署。

ISL-18 | reentrancy-no-eth

嚴重度	Medium
處置	誤報 (C)
位置	contracts/WithdrawalManager.sol:235-285

說明：Reentrancy in WithdrawalManager.processExit(uint256,address)
(contracts/WithdrawalManager.sol#235-285): External calls: -
IERC20(_pool()).transfer(owner_,redeemableShares_) (contracts/WithdrawalManager.sol#264) State variables written after the call(s): - exitCycleId[owner_] = exitCycleId_
(contracts/WithdrawalManager.sol#280) WithdrawalManagerStorage.exitCycleId
(contracts/WithdrawalManagerStorage.sol#12) can be used in cross function reentrancies: -
WithdrawalManager.addShares(uint256,address) (contracts/WithdrawalManager.sol#161-188) -
WithdrawalManagerStorage.exitCycleId (contracts/WithdrawalManagerStorage.sol#12) -
WithdrawalManager.getRedeemableAmounts(uint256,address)
(contracts/WithdrawalManager.sol#403-426) - WithdrawalManager.isInExitWindow(address)
(contracts/WithdrawalManager.sol#292-300) - WithdrawalManager.previewRedeem(address,uint256)
(contracts/WithdrawalManager.sol#319-343) - WithdrawalManager.processExit(uint256,address)
(contracts/WithdrawalManager.sol#235-285) - WithdrawalManager.removeShares(uint256,address)
(contracts/WithdrawalManager.sol#191-232) - lockedShares[owner_] = lockedShares_
(contracts/WithdrawalManager.sol#281) WithdrawalManagerStorage.lockedShares
(contracts/WithdrawalManagerStorage.sol#13) can be used in cross function reentrancies: -
WithdrawalManager.addShares(uint256,address) (contracts/WithdrawalManager.sol#161-188) -
WithdrawalManagerStorage.lockedShares (contracts/WithdrawalManagerStorage.sol#13) -
WithdrawalManager.previewRedeem(address,uint256) (contracts/WithdrawalManager.sol#319-343) -
WithdrawalManager.processExit(uint256,address) (contracts/WithdrawalManager.sol#235-285) -
WithdrawalManager.removeShares(uint256,address) (contracts/WithdrawalManager.sol#191-232) -
totalCycleShares[exitCycleId_] -= lockedShares_ (contracts/WithdrawalManager.sol#266)
WithdrawalManagerStorage.totalCycleShares (contracts/WithdrawalManagerStorage.sol#14) can be used in cross function reentrancies: - WithdrawalManager.addShares(uint256,address)
(contracts/WithdrawalManager.sol#161-188) -

WithdrawalManager.getRedeemableAmounts(uint256,address)
(contracts/WithdrawalManager.sol#403-426) - WithdrawalManager.lockedLiquidity()
(contracts/WithdrawalManager.sol#303-316) - WithdrawalManager.processExit(uint256,address)
(contracts/WithdrawalManager.sol#235-285) - WithdrawalManager.removeShares(uint256,address)
(contracts/WithdrawalManager.sol#191-232) - WithdrawalManagerStorage.totalCycleShares
(contracts/WithdrawalManagerStorage.sol#14) - totalCycleShares[exitCycleId_] += lockedShares_
(contracts/WithdrawalManager.sol#274) WithdrawalManagerStorage.totalCycleShares
(contracts/WithdrawalManagerStorage.sol#14) can be used in cross function reentrancies: -
WithdrawalManager.addShares(uint256,address) (contracts/WithdrawalManager.sol#161-188) -
WithdrawalManager.getRedeemableAmounts(uint256,address)
(contracts/WithdrawalManager.sol#403-426) - WithdrawalManager.lockedLiquidity()
(contracts/WithdrawalManager.sol#303-316) - WithdrawalManager.processExit(uint256,address)
(contracts/WithdrawalManager.sol#235-285) - WithdrawalManager.removeShares(uint256,address)
(contracts/WithdrawalManager.sol#191-232) - WithdrawalManagerStorage.totalCycleShares
(contracts/WithdrawalManagerStorage.sol#14)

判斷依據：誤報。processExit (WithdrawalManager.sol:235) 的外部呼叫是
IERC20(_pool()).transfer (:264) ，對象是本協定自己部署的 OZ ERC20 Pool ，無 transfer
hook、無回呼路徑。呼叫者另受 onlyPoolConfigurator 限制 (:241) 。

ISL-19 | uninitialized-local

嚴重度	Medium
處置	誤報 (C)
位置	contracts/LoanManager.sol:556

說明：LoanManager._advanceGlobalPaymentAccounting().accountedInterest_
(contracts/LoanManager.sol#556) is a local variable never initialized

判斷依據：誤報。uint256 accountedInterest_ (LoanManager.sol:556) 刻意以預設值 0 起算，作為 while
迴圈的累加器 (:574) ，是正確且必要的寫法。

ISL-20 | unused-return

嚴重度	Medium
處置	誤報 (C)
位置	contracts/PoolConfigurator.sol:201-210

說明：PoolConfigurator.requestRedeem(uint256,address) (contracts/PoolConfigurator.sol#201-210)
ignores return value by IPool(pool_).approve(address(withdrawalManager_),shares_)
(contracts/PoolConfigurator.sol#205)

判斷依據：誤報。IPool(pool_).approve(...) (PoolConfigurator.sol:205) 的對象是本協定自己的 OZ ERC20
Pool，approve 固定回傳 true 或 revert，不存在靜默失敗。

ISL-21 | unused-return

嚴重度	Medium
處置	誤報 (C)
位置	contracts/PoolConfigurator.sol:318-320

說明：PoolConfigurator.previewRedeem(address,uint256) (contracts/PoolConfigurator.sol#318-320) ignores return value by (None,assets_) = _withdrawalManager().previewRedeem(owner_,shares_) (contracts/PoolConfigurator.sol#319)

判斷依據：誤報。(, assets_) = _withdrawalManager().previewRedeem(...) (PoolConfigurator.sol:319) 刻意只取第二個回傳值；第一個是 redeemableShares_，在這個 view 介面上不需要。

ISL-22 | unused-return

嚴重度	Medium
處置	誤報 (C)
位置	contracts/PoolConfigurator.sol:195-198

說明：PoolConfigurator.triggerDefault(uint16) (contracts/PoolConfigurator.sol#195-198) ignores return value by (losses_,None) = _loanManager().triggerDefault(loanId_) (contracts/PoolConfigurator.sol#196)

判斷依據：誤報。(losses_,) = _loanManager().triggerDefault(loanId_) (PoolConfigurator.sol:196) 刻意忽略第二個回傳值 protocolFees_ —— 協定費已在 _handleDefault 內部處理完畢，此處只需要損失金額餵給 _handleCover。

ISL-106 | accruedInterest() 缺少到期日上限，無交易期間份額價格無界虛增

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:109-112
命中情境	L9

說明：accruedInterest_ = _getIssuance(issuanceRate, block.timestamp - domainStart) 直接用 block.timestamp 而非 min(block.timestamp, domainEnd)。貸款逾期後、下一筆狀態變更交易發生前，這個 view 函式會照原速率無上限線性累加，assetsUnderManagement() 與 Pool.totalAssets() 跟著虛增，而**申購與贖回都是按這個虛增中的價格成交**。誤差會在下一次 _advanceGlobalPaymentAccounting() 被修正，但期間已成交的申贖無法追回。

判斷依據：以 PoC 實測確認：test_PoC_Issue2_AccruedInterestGrowsPastDueDate 顯示到期日當下 986,301,368、60 天後 2,958,904,106（應凍結在前者），425 天後超過三倍以上，並同時斷言 pool.totalAssets() 一起被墊高。test_PoC_Issue2_SelfCorrectsOnNextStateChange 確認誤差在下一次結算歸零 —— 故為暫時性。上游 Maple 同一函式帶 _min(block.timestamp,

domainEnd)，移植時被移除。

建議：恢復 $\min(\text{block.timestamp}, \text{domainEnd})$ 上限。**必須同時夾 $\text{accrualEnd}_\leq \text{domainStart}_\leq$ 的下界：**修好 ISL-105 後 domainStart 會被推進到最後處理的到期日，而 domainEnd 在無其他 payment 時被設為 block.timestamp，兩者可能相等或倒置；underflow 會讓 accruedInterest() revert，而它被 assetsUnderManagement() → Pool.totalAssets() 呼叫，一旦 revert 整個池子的存提款全數卡死，後果比原缺陷更嚴重。完整 diff 見 poc/README.md 第 5.2 節。

ISL-107 | protocolFee + adminFee 合計無上限，可使 fundLoan 永久 revert

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:770-775
命中情境	L11

說明：_queuePayment 取 feeRate_ = protocolFee_ + adminFee_ (:770) 後呼叫 _getNetInterest(interest_, feeRate_)，其實作為 $\text{interest}_\times (\text{HUNDRED_PERCENT} - \text{feeRate}_) / \text{HUNDRED_PERCENT}$ (:680)。兩個費率的 setter 都沒有任何上下限：IsleGlobals.setProtocolFee(uint24) (IsleGlobals.sol:90, onlyGovernor) 與 PoolConfigurator.setAdminFee(uint24) (PoolConfigurator.sol:144, onlyAdminOrGovernor)。uint24 上限 16,777,215 相對於 $\text{HUNDRED_PERCENT} = 1\text{e}6$ 等於 1677%。任一方（或兩方合計）超過 100% 時， $\text{HUNDRED_PERCENT} - \text{feeRate}_$ 在 0.8.19 下溢 revert，fundLoan 全面失效直到費率被調回。

判斷依據：DataTypes.sol:9 的註解「uint24 adminFee; max = 1.6e7 (1600%)」顯示開發者已意識到型別容許超過 100%，但未加檢查。已確認既有 payment 不受影響 (payments[].protocolFee/adminFee 在建立時快照，而超過 100% 的組合根本無法建立成功)，故不影響還款路徑，只癱瘓新撥款。此問題與 postmortem 第 6.2 節第 6 點所述一致。

建議：在兩個 setter 加上界檢查，且必須檢查**合計**：setAdminFee 內驗證 $\text{adminFee}_+ \text{globals.protocolFee}() \leq \text{HUNDRED_PERCENT}$ ，setProtocolFee 內同理；或在 _queuePayment 以 $_min(\text{feeRate}_, \text{HUNDRED_PERCENT})$ 兜底。建議兩者都做，並把合理上限（例如 50%）寫成常數。

ISL-108 | maxCoverLiquidation 無上限，可使 triggerDefault 永久 revert

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/PoolConfigurator.sol:382-392
命中情境	L11

說明：_handleCover 計算 $\text{availableCover}_= \text{poolCover}_\times \text{_config.maxCoverLiquidation} / \text{HUNDRED_PERCENT}$ (:383)，取 $\text{coverAmount}_= _min(\text{availableCover}_, \text{losses}_)$ (:385) 後執行 $\text{poolCover}_-= \text{coverAmount}_$ (:387)。setMaxCoverLiquidation(uint24) (:154, onlyGovernor) 不檢查上限，設成大於

HUNDRED_PERCENT 時 `availableCover_ > poolCover`；只要該筆違約損失也大於 `poolCover`，`coverAmount_` 就會超過 `poolCover`，扣減時 `underflow revert` → `triggerDefault` 卡死，池子無法認列違約、無法動用 `cover`。

判斷依據：逐行確認觸發條件為 `maxCoverLiquidation > 1e6` 且 `losses_ > poolCover`；後者在無擔保信貸池是常態（postmortem 記載 53 筆已還款中 52 筆逾期）。由 `governor` 誤設觸發，非外部可控，但後果是違約流程整體不可用，且發生時正是最需要它的時刻。

建議：在 `setMaxCoverLiquidation` 加 `if (maxCoverLiquidation_ > HUNDRED_PERCENT) revert ...`；並在 `_handleCover` 以 `_min(availableCover_, poolCover)` 兜底。

ISL-109 | `Receivable.createReceivable` 完全無存取控制

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/Receivable.sol:59-76
命中情境	L2

說明：`createReceivable (:59)` 為 `external` 且無任何 `modifier`，任何地址都能鑄造一張 `buyer / seller / faceAmount / repaymentTimestamp` 全部自填的應收帳款 NFT，並 `_safeMint` 到自己指定的 `params_.seller` 錢包 (:72)。 `Receivable.Info.isValid` 一律寫死 `true (:69)`，合約端無任何真實性驗證。

判斷依據：已逐行確認放款路徑本身沒有被打穿：`LoanManager.requestLoan` 要求 `msg.sender == receivableInfo_.buyer (:194、:943-947)`，且 `_revertIfInvalidReceivable (:949-965)` 另外驗證 `poolConfigurator_.buyer() == buyer_` 與 `isSeller(seller_)`，撥款還需 `onlyPoolAdmin`。因此攻擊者無法自助取得貸款。實際影響有三：(a) 任何人可把 NFT 塞進任意賣方錢包，污染以此 NFT 為準的鏈下對帳；(b) `_tokenIdCounter` 可被無成本灌大；(c) 這是 D-RWA-01（鏈下事實無法被合約驗證）在本專案最直接的體現——「一張應收帳款存在」這個宣稱在鏈上不需要任何憑據。

建議：把 `createReceivable` 限制為白名單角色（`governor`、`pool admin`，或 `isSeller` 名單），或改為需要買方簽章授權。長期建議引入 `attestation` 機制讓外部可驗證憑證對應真實發票。

ISL-110 | 借款人自訂的 `gracePeriod` 無上限，可使 `triggerDefault` 永久無法觸發

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:173-228
命中情境	L11

說明：`requestLoan` 由買方（借款人）呼叫，`gracePeriod_`、`rates_[0]`（利率）與 `rates_[1]`（逾期加成）全部是呼叫者自填的參數 (:176-178)，合約不設任何上下限，直接寫進 `_loans[loanId_] (:211-225)`。而 `gracePeriod` 又被用在 `triggerDefault` 的前置檢查：`if (block.timestamp <= _loans[loanId_].dueDate + _loans[loanId_].gracePeriod)`

revert (:427)。借款人填入極大值（或使該加法溢位的值）即可讓違約流程**永久無法執行**——極大值使條件恆真而 revert，溢位值則直接算術 revert，兩條路都讓出借方失去違約工具。

判斷依據：已確認唯一的把關是 fundLoan 的 onlyPoolAdmin 人工審核；但 pool admin 在核准時看到的是一份參數清單， $\text{gracePeriod} = 2^{255}$ 這種值不必然會被注意到，而鏈上沒有任何自動阻擋。此模式為本次領域調查新歸納的 D-CREDIT-02（借款人自訂參數被用於保護出借方的檢查），詳見 audit/DOMAIN_RESEARCH.md 第 3 節。

建議：在 requestLoan 對 gracePeriod_ 設合理上限（例如 90 days），對 rates_ 兩個元素設上下限；或改為由 pool admin 在 fundLoan 時指定這三個參數，讓借款人只能提出申請、不能決定保護條款。

ISL-112 | setExitConfig 的 cycleDuration 無上限，pool admin 可實質凍結全部贖回

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/WithdrawalManager.sol:105-158
命中情境	L11

說明：setExitConfig (:105, onlyAdminOrGovernor) 只驗證 windowDuration_ != 0 (:114) 與 windowDuration_ <= cycleDuration_ (:118)，cycleDuration_ 本身為 uint64 且無上限。pool admin 可把週期設成數十年；新設定在三個 cycle 後生效 (:125)，之後所有 getCurrentCycleId() 與 getWindowAtId() 都依新週期計算，既有與後續的提領請求都會被推到極遠的未來，而系統沒有任何緊急贖回或治理否決的繞道。

判斷依據：逐行確認無其他路徑可取回資產：Pool.withdraw 直接 revert Pool_WithdrawalNotImplemented (:193)，redeem 必經 maxRedeem → isExitWindow (PoolConfigurator.sol:310-315)。removeShares 可取回 pool share，但那只是換回份額憑證、不是換回底層資產。此模式為本次領域調查新歸納的 D-CREDIT-03，詳見 audit/DOMAIN_RESEARCH.md 第 3 節。

建議：對 cycleDuration_ 設上限（例如 90 days）與下限；並考慮加入「設定變更不得延後既有 pending 請求的既定視窗」的保護。

ISL-113 | 升級與位址改寫權限集中於單一 governor 位址，合約層無多簽或 timelock 要求

嚴重度	Medium
處置	已知風險但可接受 (B)
位置	contracts/PoolAddressesProvider.sol:145-179

說明：Governable 只維護單一 address governor (Governable.sol:15)，兩段式移轉但無 timelock、無門檻、無第二人核准。該位址可：(a) setAddress(LOAN_MANAGER, x) (PoolAddressesProvider.sol:145) 把 loan manager 換成任意地址——由於 Pool 建構子已對 PoolConfigurator 授權 type(uint256).max (Pool.sol:36)，而 requestFunds 只檢查 msg.sender ==

loanManager_ (PoolConfigurator.sol:175) ，該地址即可一次提走 pool 全部流動性；(b) setLoanManagerImpl / setPoolConfiguratorImpl / setWithdrawalManagerImpl / setAddressAsProxy 替換任一核心 implementation ；(c) setIsleGlobals 連同 governor 自身一起換掉；(d) 升級 IsleGlobals 與 Receivable (UUPS) 。

判斷依據：此為 audit/DOMAIN_RESEARCH.md 的 D-RWA-02（來源：rwa.md，last_reviewed 2026-08-12），該條記載 2025 年 RWA 領域最大宗事故即為單一 deployer 私鑰被取得後呼叫 upgradeToAndCall 替換 implementation，損失約 850 萬美元。本專案的存取控制**實作正確**（已逐一確認 onlyGovernor 的比較對象與 IsleGlobals.governor() 一致，無 tx.origin、無恆真條件），問題在於權力邊界本身。列為 B 而非 A，因為這是明確的產品設計選擇而非程式缺陷——但緩解措施屬於部署與營運層，必須在鏈下落實並向存款人揭露。

ISL-114 | 升級的原子性完全依賴呼叫者傳入正確的 params，否則 initialize 對任何人開放

嚴重度	Medium
處置	已確認需修復 (A)
狀態	待處理
位置	contracts/libraries/upgradability/VersionedInitializable.sol:37-60
命中情境	L7

說明：VersionedInitializable.initializer 的放行條件是 revision > lastInitializedRevision (:45)，屬 revision 制而非一次性 flag。PoolAddressesProvider._updateImpl 雖然呼叫 upgradeToAndCall(newAddress, params) (:177)，但 params 是 setXxxImpl 的呼叫者自帶參數，**合約不驗證它非空、也不驗證它確實是一個 initializer 呼叫**。若 governor 以空 params 執行一次會提高 revision 的升級，proxy 會停在「implementation 已換、initialize 尚未執行」的狀態，此時任何人都能搶先呼叫 initialize —— 而三份合約的 initialize 都沒有呼叫者檢查：LoanManager.initialize(address asset_) (LoanManager.sol:53) 可寫入任意 asset；PoolConfigurator.initialize (:76) 與 WithdrawalManager.initialize (:67) 只比對 provider_ 是否等於 immutable 的 ADDRESSES_PROVIDER，而那是攻擊者可以照填的公開值。

判斷依據：已確認**目前尚未可利用**：三份合約的 revision 皆為 0x1，代理上的 lastInitializedRevision 已是 1，revision > lastInitializedRevision 為假，再次呼叫 initialize 會 revert。風險在下一次升級 —— 而 postmortem 第 6.1 節第 2 點規劃的 ISL-105/106 修復正是一次 revision 升到 2 的升級，屆時此窗口會真實存在。這是「修復缺陷的那次操作本身帶著新風險」的典型情況，必須在修復前先處理。

建議：在三份合約的 initialize 加入呼叫者檢查 (msg.sender == address(ADDRESSES_PROVIDER) 或 proxy admin)，使搶跑不可行；並在 setXxxImpl 系列函式要求 params.length != 0，把「必須原子升級」從營運紀律變成合約強制。

ISL-111 | 同一張應收帳款可支撐多筆貸款，合約無「已使用」標記

嚴重度	Low
-----	-----

處置	已確認需修復 (A)
狀態	待處理
位置	contracts/LoanManager.sol:173-249
命中情境	L2

說明：requestLoan 只讀取 getReceivableInfoById (:190-191)，不檢查該 receivablesTokenId_ 是否已被其他貸款引用，也不在 receivable 上留下任何消耗標記。同一個買方可用同一張 NFT 重複申請，pool admin 若重複撥款，requestFunds 會把兩筆本金都從 pool 轉出並計入 principalOut。

判斷依據：已確認損失被第二道機制部分擋住：withdrawFunds 要求 safeTransferFrom(msg.sender, address(this), tokenId) (:303)，第一筆提領後 NFT 已易主，第二筆無法提領，資金會滯留在 LoanManager。因此不是直接盜取，而是「資金離開 pool、計入 principalOut、但無對應可提領債權」的帳務失真，且需要 pool admin 重複撥款才會發生 → 評為 Low。

建議：在 requestLoan 檢查該 tokenId 是否已有未結清的貸款（新增 mapping(address => mapping(uint256 => uint16)) activeLoanOfReceivable），或要求申請時即把 NFT 托管給 LoanManager。

ISL-115 | 無儲備證明機制：流通份額對應的債權無法被外部獨立驗證

嚴重度	Informational
處置	已知風險但可接受 (B)
位置	contracts/Receivable.sol:59-92

說明：系統的資產淨值 _totalAssets() = pool 餘額 + assetsUnderManagement() (PoolConfigurator.sol:344)，其中 assetsUnderManagement() 完全由鏈上記帳推導 (principalOut + accountedInterest + accruedInterest())。但這些數字對應的**債權是否真實存在**，鏈上沒有任何驗證或證明機制：Receivable.Info.isValid 建立時寫死 true、createReceivable 無權限 (ISL-109)、無 attestation 函式、無鏈下儲備報告串接點、無稽核時點事件。

判斷依據：對照 audit/DOMAIN_RESEARCH.md 的 D-RWA-05 (來源：rwa.md) 四個標準查證問題，四項皆為否。本次 postmortem 正是這個缺口的實證：accountedInterest 累積了 \$3,281 沒有任何真實債權對應，而系統本身沒有任何機制會發現——是靠人工重放鏈上事件、與獨立重算的應計總和比對才查出來的。列為 Informational 是因為這是機制缺失而非可被直接利用的漏洞，但它決定了「下一次帳對不上時要多久才會現形」。

已評估項目摘要

以下 82

項為低嚴重度（Low／Informational）且經判定為可接受風險（B）或誤報（C）的發現，依檢查器與判定理由歸併為 73 組。逐筆明細保留於工作底稿，可依需要調閱。

檢查器	筆數	處置	判定理由
naming-convention	8	誤報（C）（自動預分類）	純命名/事件索引等風格檢查（naming-convention），不構成安全性問題，由 scan 依 STYLE_ONLY_CHECKS 自動預分類為 C。如認為此筆有安全含義，清...
unindexed-event-address	3	誤報（C）（自動預分類）	純命名/事件索引等風格檢查（unindexed-event-address），不構成安全性問題，由 scan 依 STYLE_ONLY_CHECKS 自動預分類為 C。如認為此筆有...
shadowing-local	1	誤報（C）	誤報。UUPSProxy 建構子參數 _implementation（libraries/upgradability/UUPSProxy.sol:7）遮蔽 OZ `ERC1...
missing-zero-check	1	已知風險但可接受（B）	屬實但可接受。setBuyer(address(0))（PoolConfigurator.sol:129-131）不會造成資金風險，只會讓 `_revertIfInvalidR...
missing-zero-check	1	誤報（C）	誤報。pool_ 來自同一函式內 PoolDeployer.createPool 的回傳值（PoolConfigurator.sol:104-105），`new Pool(...
reentrancy-benign	1	誤報（C）	誤報。depositCover（PoolConfigurator.sol:246-250）在 safeTransferFrom 之後才 `poolCover += amou...
reentrancy-benign	1	誤報（C）	誤報。Pool._deposit（Pool.sol:287-299）先轉帳後 mint 是 OZ ERC4626 的標準寫法，原始碼 :288-294 有完整註解說明此順序刻意...
reentrancy-benign	1	已知風險但可接受（B）	屬實但可接受。withdrawCover（PoolConfigurator.sol:253-265）先 safeTransfer (:256) 後 `poolCover -= ...
reentrancy-benign	1	誤報（C）	誤報。initialize（PoolConfigurator.sol:76-112）的外部呼叫是 PoolDeployer.createPool (:104-105)，`ne...

reentrancy-benign	1	誤報 (C)	誤報 (與 ISL-02 同一函式的 benign 變體)。此筆指的是事件與狀態變數的相對順序，真正需要處理的重入面向已在 ISL-02 列為必須修復，修法相同，不重複計列。
reentrancy-benign	1	誤報 (C)	誤報。同 ISL-16，fundLoan 已有 nonReentrant，Slither 無法辨識自製 guard。
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...

reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...

reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra...

reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra…
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra…
reentrancy-events	1	誤報 (C)	誤報。reentrancy-events 只指出「事件在外部呼叫之後才發出」，後果限於鏈下索引器可能觀察到亂序事件，不影響任何鏈上狀態或資金。本專案的事件消費者為自有 subgra…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…

timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…

timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…

timestamp	1	誤報 (C)	誤報。本專案是天期以「日」計的應收帳款融資協定（實測貸款天期中位數 8.3 天、逾期中位數 20.6 天），礦工可影響的時間戳偏移量（秒級）比任何具經濟意義的門檻小上四個數量級，無…
assembly	1	誤報 (C)	誤報。libraries/ReentrancyGuard.sol:19-23 的 assembly 只做 ERC-7201 命名空間 storage 的 slot 定位（`\$.…
assembly	1	誤報 (C)	誤報。VersionedInitializable.is Constructor() (:81-93) 用 extcodesize(address()) 判斷是否在建構子中，是 …
assembly	1	誤報 (C)	誤報。_getInitializableStorage() (:95-99) 同 ISL-76，ERC-7201 slot 定位。
pragma	1	已知風險但可接受 (B)	屬實但可接受。專案自身 33 個檔案全部固定 pragma solidity 0.8.19，版本分歧來自 modules/ 底下 OZ v4.9.2 的 ^0.8.0 …
dead-code	1	已知風險但可接受 (B)	屬實。_updateImpl(bytes32, address) 兩參數版本（PoolAddressesProvider.sol:162-164）確實無人呼叫，全部入口都走三參數…
dead-code	1	已知風險但可接受 (B)	屬實。_getLastInitializedRevision() (VersionedInitializable.sol:101-103) 為 Aave 原始碼保留的 intern…
solc-version	1	已知風險但可接受 (B)	屬實但可接受。已逐一比對 Slither 指出的三個 0.8.19 已知問題對本專案是否成立：(a) VerbatimInvalidDeduplication —— 僅影響 Yul…
naming-convention	1	誤報 (C) (自動預分類)	純命名風格（library 名為 CapWords，Slither 期望 mixedCase）。無安全影響。但要特別記錄：library PoolConfigurator 與 …
naming-convention	1	誤報 (C) (自動預分類)	純命名風格。同 ISL-84：library Loan 與 LoanManager 的型別引用構成 Slither 名稱解析衝突的一環。
naming-convention	1	誤報 (C) (自動預分類)	純命名風格。同 ISL-84：library Receivable 與 contract Receivable 同名，是 Slither 解析失敗的直接觸發點之一。

redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
redundant-statements	1	誤報 (C)	誤報。這是 Solidity 用來抑制「未使用參數」警告的慣用寫法 —— Pool.withdraw / previewWithdraw / maxWithdraw 是…
immutable-states	1	已知風險但可接受 (B)	屬實但不修。Pool.configurator (Pool.sol:20) 確實只在建構子寫入、之後從未變更，改成 immutable 可省下每次讀取的 SLOAD。不在本次處…

附錄：發現處置分類

本報告對每一筆發現標示兩個獨立欄位：**嚴重度**（Critical／High／Medium／Low／Informational，由工程團隊依實際影響判定）與**處置**（下列 A／B／C／D）。掃描工具自身回報的 impact 若與本報告呈現的嚴重度不同，該筆會並列印出兩者與調整理由。

- **A | 已確認需修復**：確認為真實問題（多涉及資金流向或權限控制邏輯），必須修復。
- **B | 已知風險但可接受**：問題確實存在，但經工程團隊評估風險可控（例如僅管理者可呼叫、另有其他層級防護），附具體理由後接受並於報告揭露。
- **C | 誤報**：靜態分析限制造成的誤判，實際已有防護機制或該判斷邏輯不適用。
- **D | 待確認**：尚無法判定歸屬者一律列此類；判讀信心不足時寧列 D，不猜測分類。此類項目均附「要確認什麼／由誰確認／兩種答案各自的處置」。
- 人工複核發現僅得分類 A／B／D；經確認非問題者直接自清單移除，不設誤報分類。